

CinemaSeat

Database design & API documentation — Zero to Production Phase 2, 8 August 2026

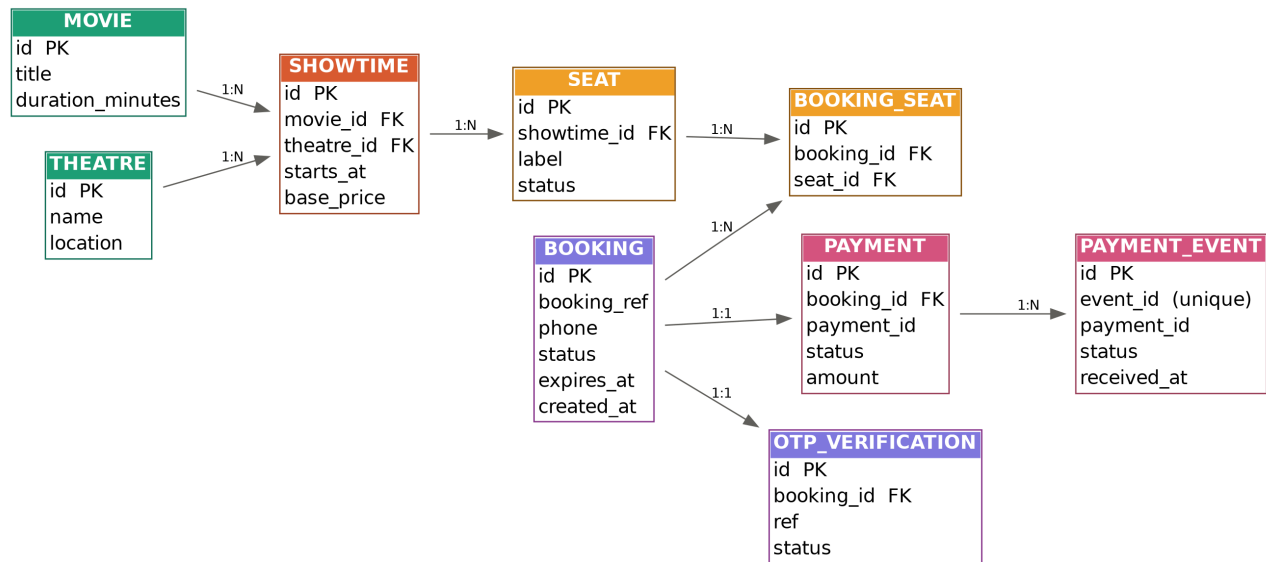
Modular monolith: one Django REST Framework service, one PostgreSQL database, integrated with the provided mock payment/OTP gateway. Not split into microservices — nothing here has an independent scaling need yet, and splitting adds deployment complexity that isn't justified in an 8-hour build. If this system grew, the payment webhook module is the most likely candidate to split off first, since it's the one piece that could reasonably become an async queue consumer without touching the seat-hold logic.

Architecture



Seat hold and booking is the module the concurrency scenarios (Milestone 4, Scenario A) directly test — 100 concurrent hold requests on one seat, exactly one must succeed, ninety-nine must be cleanly rejected, zero oversell.

Database design (ERD)



Key design decisions

BOOKING_SEAT is a join table, not a direct FK on Seat. A booking can cover multiple seats. The concurrency lock still operates per-seat: each requested seat is locked individually, in a deterministic order (sorted by seat id), to avoid deadlocks when two multi-seat bookings request overlapping seats in opposite order.

PAYMENT_EVENT exists purely as the idempotency guard. event_id is unique. On webhook receipt: get_or_create(event_id=...). If it already existed, skip all state changes and return 200 anyway — this is the entire fix for "a duplicate callback must not confirm a booking twice."

Payment and Booking are separate tables, not merged. A booking can go through failed → retried payment attempts. Keeping Payment as its own row per attempt (never overwritten) preserves the audit trail Milestone 4's "prove it" scenarios will want to see.

No User table — phone number is the identity. OTP verification against the gateway is the identity check. A full account system (passwords, sessions) is unneeded scope here, unlike a system with persistent user profiles.

API reference

Method	Endpoint	Description
GET	/api/health/	Liveness + readiness, <1s even if gateway is down
GET	/api/movies/	List movies
GET	/api/showtimes/?movie_id=	List showtimes, optionally filtered
GET	/api/showtimes/{id}/seats/	Live seat map for a showtime (judged)
POST	/api/bookings/hold/	Hold one or more seats (judged)
POST	/api/bookings/{id}/otp/send/	Trigger OTP to the phone on the booking
POST	/api/bookings/{id}/otp/verify/	Verify the OTP code
POST	/api/bookings/{id}/pay/	Kick off payment, returns immediately
POST	/api/webhooks/payment/	Gateway calls this back, always 200, idempotent
GET	/api/bookings/{id}/	Poll booking/payment status

Judges point automated tests directly at the two endpoints marked "(judged)" above — these exact request/response shapes must be implemented precisely as documented.

Fetch a seat map — exact request

```
GET /api/showtimes/{showtime_id}/seats/
200 OK
[
  { "id": 12, "label": "F12", "status": "AVAILABLE", "price": 450 },
  { "id": 13, "label": "F13", "status": "HELD", "price": 450 },
  { "id": 14, "label": "F14", "status": "BOOKED", "price": 450 }
]
```

Hold a seat — exact request

```
POST /api/bookings/hold/
Content-Type: application/json

{
  "showtime_id": 7,
  "seat_ids": [12],
  "phone": "+8801XXXXXXXXX"
}
```

Success:

```
201 Created
{
  "booking_id": "bk_9f3a",
  "status": "HELD",
}
```

```
    "expires_at": "2026-08-08T14:32:10Z",  
    "seats": [{ "id": 12, "label": "F12" }]  
  }
```

Seat already taken/held — this is what Scenario A hammers:

```
409 Conflict  
{ "error": { "detail": "One or more seats are no longer available." } }
```

Payment webhook — exact contract (from the gateway spec)

```
POST /api/webhooks/payment/  
Content-Type: application/json
```

```
{  
  "event_id": "evt_001",  
  "payment_id": "pay_xyz",  
  "booking_ref": "bk_9f3a",  
  "status": "SUCCEEDED",  
  "amount": 450  
}
```

Always, regardless of new / duplicate / malformed — a non-200 makes the gateway retry forever:

```
200 OK
```

Documented gateway misbehaviour (design for this, don't mock it away)

Behaviour	Rate
Callback delayed 2–15 seconds	Always
Payment fails (FAILED)	10%
Same callback delivered twice	8%
/charge returns 500 or times out	2%
OTP delayed or never delivered	10%

Build against X-Mock-Mode: deterministic first (2s delay, always succeeds, no duplicates), then switch to real chaos before trusting anything works.